



Compte rendu de PSTL

Analyse statique avec MOPSA de programmes
eBPF traduits en C

Encadrant : Miné Antoine

Etudiant 1 : ZUHAIR Saïd Mohammad 21204924

Etudiant 2 : RALLE Gérald 21312745

Etudiant 3 : HABA Abdallah 3812460

1. Contexte du projet.....	3
1.1. eBPF.....	3
1.2. MOPSA.....	3
2. Cahier des charges.....	4
2.1. Objectifs du projet.....	4
2.2. Fonctionnalités.....	4
2.3. Etapes du projet.....	5
2.4. Travaux Liés.....	5
3. Tâches réalisées.....	6
3.1. Mise au point du programme d'entrée.....	6
3.2. Parser du code octet.....	6
3.3. Ecriture du programme C.....	7
3.4. Ecrire une bibliothèque des fonctions eBPF.....	10
3.5. Problèmes rencontrés.....	11
3.5.1. Tableaux statiques.....	11
3.5.2. Maps eBPF.....	12
3.6. Préparation des exemples.....	13
3.7. Exemple.....	14
3.8. Analyse avec MOPSA.....	16
3.8.1. Banc de tests.....	16
3.8.2. Analyse.....	16
4. Réalisations additionnelles.....	18
4.1. Prise en charge des structures courantes.....	18
4.2. Amélioration des tableaux eBPF.....	18
5. Perspectives d'améliorations pour le futur.....	19
5.1. Pointeurs.....	19
5.2. Gestion de la taille des entiers.....	19
5.3. Étendre la bibliothèque.....	20
6. Conclusion.....	21
7. Bibliographie.....	21

1. Contexte du projet

Dans le cadre de notre projet STL, nous avons développé un système visant à vérifier la sûreté des programmes eBPF (Extended Berkeley Packet Filter) utilisés dans le noyau Linux. Pour ce faire, nous avons mis en place un traducteur capable de convertir le code-octet eBPF en langage C, ainsi qu'une bibliothèque simulant l'environnement d'exécution de ces programmes, afin de permettre leur analyse par l'outil MOPSA.

1.1. eBPF

Extended Berkeley Packet Filter (eBPF) est une machine virtuelle intégrée au cœur du système d'exploitation Linux, permettant l'exécution de programmes arbitraires. À l'origine destinée à l'analyse, au filtrage et à la transformation des paquets réseau, eBPF est désormais utilisé dans une gamme variée d'applications nécessitant une proximité avec le noyau et un accès à des données privilégiées, tout en évitant les transferts de données vers les processus utilisateur pour des raisons d'efficacité. Les programmes eBPF sont définis dans un code-octet spécifique, comportant un ensemble restreint d'instructions, de registres, et offrant un accès de bas niveau et non typé à la mémoire (semblable à de l'assembleur). Ils peuvent être chargés dynamiquement pour étendre les fonctionnalités du noyau.

1.2. MOPSA

Le LIP6 développe un analyseur statique de programmes appelé MOPSA ayant pour but de détecter les erreurs de programmation simples, comme les débordements de tableau ou les erreurs de pointeurs.

L'analyseur prend en entrée les sources du programme, effectue un calcul qui s'apparente à une interprétation dans un domaine non-standard, pour en déduire une représentation symbolique de toutes ses exécutions possibles. Afin d'être efficace, il emploie des représentations approchées des exécutions. Une particularité est que l'outil se veut sûr : l'approximation représente systématiquement un sur-ensemble des comportements possibles, pour être certain de ne pas en oublier. L'outil est pessimiste et peut indiquer des erreurs où, en réalité, il n'y en a pas (fausses alarmes). Par contre, il ne doit pas oublier d'erreur possible.

2. Cahier des charges

2.1. Objectifs du projet

Notre objectif est de permettre la vérification de programmes eBPF avec MOPSA. Or, MOPSA ne peut pas lire les programmes eBPF, ils sont en code-octet. Étendre MOPSA pour supporter directement eBPF serait possible, mais assez complexe. Par ailleurs, eBPF étant un langage de très bas niveau, il semble plus adapté de le transformer en C, plutôt qu'en Python (déjà compatibles avec MOPSA).

Le projet implique tout d'abord une phase de documentation approfondie sur les technologies pertinentes, notamment eBPF et MOPSA, suivie du développement incrémental d'un traducteur eBPF vers C pour permettre l'analyse statique avec MOPSA, en se concentrant sur la traduction précise des instructions eBPF et la vérification régulière avec des exemples de programmes. En parallèle, une recherche et sélection de programmes eBPF réalistes est effectuée pour une évaluation exhaustive, tandis qu'une bibliothèque C est développée pour simuler l'environnement d'exécution des programmes eBPF, assurant ainsi une analyse précise avec MOPSA en combinant le code traduit et l'environnement simulé. Il est à noter que le langage choisi pour implémenter le convertisseur d'eBPF vers C de ce projet est Java, nécessitant une adaptation des outils de manipulation d'eBPF et des approches pour s'aligner avec cette décision de développement. L'environnement d'analyse des programmes eBPF est écrit en C, car il s'agit du langage analysé par MOPSA, c'est pourquoi il s'agit du langage cible de notre transpileur. Quant à MOPSA, il a été écrit en OCaml, mais nous ne nous intéresserons pas au code de cet outil dans ce projet, car il n'est pas à modifier.

2.2. Fonctionnalités

Les fonctionnalités attendues pour ce projet comprennent le développement d'un traducteur eBPF vers C et d'une bibliothèque C simulant l'environnement eBPF. Le traducteur devra être capable de convertir de manière précise et complète le code-octet eBPF en code C équivalent, avec une documentation détaillée accompagnant son fonctionnement et sa validation. De son côté, la bibliothèque de simulation devra fournir des fonctions et des structures de données fidèlement représentatives des interactions entre les programmes eBPF et le noyau Linux. L'objectif est d'assurer une compatibilité et une intégration réussies avec MOPSA,

tout en permettant des analyses statiques exhaustives pour détecter les erreurs potentielles dans les programmes eBPF.

MOPSA ne peut analyser que des programmes complets, c'est-à-dire des programmes qui comportent un point d'entrée (main) et aucune fonction non-définie. Si un programme eBPF fait un appel au noyau, il faut fournir à MOPSA une modélisation de la fonction appelée (i.e., une fonction en C simulant de manière abstraite les comportements possibles de la fonction, en particulier son impact sur la mémoire visible par le programme eBPF). De même, si le programme eBPF utilise des variables globales du système, il faut s'assurer qu'elles sont initialisées avant d'analyser le point d'entrée de l'analyse. Les différents types d'erreurs que MOPSA est capable de vérifier sont les erreurs d'exécution (débordements dans la mémoire, opérations arithmétiques ou de pointeur non valides, accès à la mémoire non valides, doubles libérations de mémoire), les échecs d'assertion et les échecs de conditions préalables pour les appels à la bibliothèque C (arguments non valides, formats non valides, ...).

2.3. Etapes du projet

Le projet se déroulera en plusieurs étapes clés. Tout d'abord, une phase de documentation approfondie sur eBPF, MOPSA et le développement en Java sera entreprise pour acquérir une compréhension approfondie des technologies impliquées. Ensuite, le développement du traducteur eBPF vers C et de la bibliothèque de simulation sera entreprise, avec un focus sur la précision et la complétude des traductions ainsi que sur la fidélité de la simulation. Par la suite, l'intégration des composants avec MOPSA et des tests approfondis seront réalisés pour garantir leur compatibilité et leur efficacité. Enfin, le projet sera finalisé avec la rédaction des rapports et la présentation des résultats.

2.4. Travaux Liés

Les deux projets intéressants liés à notre projet sont le projet PREVAIL et le vérificateur du code eBPF du noyau.

L'outil PREVAIL est basé sur des principes formels de vérification du logiciel, c'est-à-dire qu'il utilise des techniques mathématiques pour prouver que le code respecte un ensemble de propriétés de sécurité.

Le noyau Linux, quant à lui, dispose aussi d'un vérificateur eBPF intégré, qui sert à s'assurer que le code eBPF est sûr avant d'être exécuté dans le noyau. Le vérificateur

analyse le code eBPF, en contrôlant la validité des instructions, si elles respectent un certain nombre de politiques de sécurité, etc.

3. Tâches réalisées

3.1. Mise au point du programme d'entrée

Nous utilisons la bibliothèque libbpf et libbpf-bootstrap pour générer des exemples de code octet eBPF. Les exemples compilés avec ces bibliothèques sont en format ELF (Executable and Linkable File). Les fichiers ELF contiennent plusieurs sections qui correspondent aux différents aspects d'un programme exécutable. Les deux sections que nous utilisons dans notre projet sont la section des chaînes de caractères et la section du code octet du programme compilé. Nous utilisons un fichier "sectionName" dans lequel on écrit le nom de la section de code octet du fichier ELF et à l'aide de la bibliothèque que nous utilisons pour charger le fichier binaire ELF, nous récupérons le code octet du programme source. Pour récupérer les données des chaînes de caractère nous utilisons un script qui appelle la commande "[objcopy](#)" et copie la section `".rodata"` du fichier ELF dans un autre fichier qu'on appelle "strings".

3.2. Parser du code octet

Les instructions de eBPF sont codées en 64 bits selon la structure suivante:

32 bits (bit de poids fort)	16 bits	4 bits	4 bits	8 bits
immédiate	offset	registre source	registre destination	opcode

- Opcode : C'est le code de l'opération. Il s'agit du champ qui spécifie l'opération à effectuer, comme l'ajout, la soustraction, la multiplication, etc.
- Les registres source et destination : ces champs désignent les registres qui seront utilisés pour les opérations. Les instructions eBPF travaillent généralement avec des registres plutôt qu'avec des valeurs en mémoire. Ces registres peuvent être considérés comme l'équivalent des variables en eBPF. Ils peuvent servir d'arguments de fonctions, sans quoi il ne serait pas

possible de faire appel à ces dernières.

- Des champs supplémentaires pour les constantes ou les décalages : certaines instructions peuvent nécessiter des valeurs constantes ou des décalages pour effectuer leur travail.
- Des marqueurs de contrôle de flot d'instruction : certains champs peuvent être utilisés pour spécifier des conditions de branchement ou de saut, ce qui permet d'implémenter des structures de contrôle comme les boucles et les conditions.

Nous avons d'abord créé un parser pour ces instructions. Ensuite nous avons comparé la représentation textuelle du code eBPF généré par notre parser avec celle de la bibliothèque ebpf-disasm (désassembleur du code eBPF basé sur rbpf qui est un interprète du code eBPF écrit en Rust) pour vérifier le bon fonctionnement de notre parser. Nous utilisons aussi le programme bpftool qui permet d'afficher les programmes eBPF après qu'ils soient chargés dans le noyau et passés par le vérificateur de ce dernier, pour vérifier que notre parser marche correctement.

3.3. Ecriture du programme C

eBPF contient 11 registres:

Registre	Rôle
r0	Contient la valeur du retour des fonctions et du programme.
r1 - r5	Contient les paramètres des appels de fonctions. Au début du programme, il contient le "contexte" du programme.
r6 - r9	Contient des valeurs qui ne sont pas changées après les appels de fonctions. Ce sont effectivement des variables globales.
r10	Pointeur de la pile.

Notre traducteur convertit les registres eBPF jouant le rôle de variables en véritable variable dans le code C généré. Ceux-ci sont gérés par une HashMap dans notre projet. Le type des variables dans notre programme en sortie est "uint64_t" car la plupart des opérations de eBPF se font sur des valeurs de taille 64 bits. La boucle de notre traducteur eBPF vers C se passe de la manière suivante :

→ Récupération de l'instruction :

La première étape consiste à extraire l'instruction à traiter à partir du programme source. Cette étape est réalisée à l'aide du parser qui prend en entrée le code eBPF et stocke le résultat dans des records qu'on a défini en Java.

→ Vérification de l'existence des variables :

Avant de créer de nouvelles variables, il faut vérifier si des variables correspondantes existent déjà dans la liste des variables. Cette vérification préliminaire permet d'éviter la duplication des données et de garantir la cohérence du programme.

→ Gestion des variables :

- ◆ Si une variable correspondante est déjà présente dans la liste des variables, nous récupérons son identifiant pour une utilisation ultérieure.
- ◆ En l'absence de variables correspondantes, nous créons de nouvelles variables pour les registres source et destination. Ces nouvelles variables contiennent les adresses nécessaires et sont ajoutées à la liste des variables pour être utilisables dans la suite du programme.

→ Création de labels :

Chaque instruction nécessite un label pour marquer son emplacement dans le code. Les labels permettent de référencer facilement ces emplacements à d'autres endroits du programme. Ainsi, nous créons un label pour chaque instruction afin de faciliter la navigation et la gestion du code. Ces labels sont surtout importants pour les instructions de sauts, conditionnelles ou non.

→ Traduction en code C :

Une fois les vérifications et les préparatifs terminés, nous traduisons l'instruction eBPF récupérée en code C équivalent. Notre code Java permet, via un switch/case, de couvrir tous les opcodes des instructions possibles d'eBPF, et renvoie en sortie le code C correspondant à une instruction donnée.

Après avoir traité toutes les instructions, nous écrivons d'abord la partie des chaînes de caractères statiques du fichier ELF d'entrée dans une variable globale, puis, dans la fonction main, nous écrivons les déclarations des variables locales et ensuite le code généré du programme.

Nous avons remarqué que dans le code eBPF, un entier est utilisé à la place des chaînes de caractères passées en paramètre : il s'agit de l'indice du début de la chaîne de caractères dont nous avons besoin pour l'appel de la fonction. Nous en avons donc logiquement conclu que les chaînes de caractères statiques d'un programme eBPF ne sont référencés que par leurs indices. Ainsi, nous avons décidé de traiter ce cas séparément en utilisant cet indice en rapport avec la variable globale pour gérer ce cas.

Exemple :

Nous avons ici un exemple de code simple généré :

```
char * strings = "facto %d: %d\0";

int main() {
    __u64 r0;
    __u64 r1;
    __u64 r2;
    __u64 r3;
    __u64 r4;
L0: r0 = (__u64) bpf_get_current_pid_tgid();
L1: r3 = 0;
L2: if (r0 == 0) goto L9;
L3: r3 = r0;
L4: r3 += -1;
L5: r1 = r3;
L6: r1 *= r0;
L7: r0 = r1;
L8: if (r1 != 0) goto L4;
L9: r1 = 0;
L10: r2 = 13;
L11: r4 = 0;
L12: bpf_trace_printk((char*)(strings + r1), (__u32) r2, r3);
L13: r0 = 0;
L14: return r0;
}
```

On peut remarquer que dans cet exemple

- **L0-L14** représentent les labels de chaque instruction.
- L'instruction **goto** permet de sauter directement à l'une d'elle avec son nom. On peut ainsi réaliser des boucles suivant certaines conditions.
- **r0-r4** sont les variables représentant les registres eBPF.
- **strings** est la variable globale contenant les chaînes de caractères de tout le programme.
- Et **bpf_get_current_pid_tgid** et **bpf_trace_printk** sont des appels de fonctions de notre bibliothèque d'exécution.

3.4. Ecrire une bibliothèque des fonctions eBPF

Pour représenter une fonction en C nous utilisons un record de Java que l'on a appelé CFunction. Cela nous permet de facilement intégrer ces fonctions dans les instructions `call` d'eBPF mais aussi de générer la bibliothèque des fonctions à partir d'une même base.

Ces fonctions sont ajoutées dans une HashMap en Java. Afin de garantir leur unicité, les clés correspondent à l'adresse à partir de laquelle les instructions `call` d'eBPF appellent ces fonctions (trouvables dans le manuel d'eBPF). Quant aux valeurs, elles correspondent à une définition superficielle de la fonction correspondante, définie par la classe CFunction.

Une instance de CFunction est définie par les 5 attributs suivants :

- **returnType :**

Il représente le type de retour de la fonction helper d'eBPF. Il permet de déterminer comment le résultat de la fonction sera interprété par le reste du programme eBPF.

- **name :**

Il s'agit du nom de la fonction helper. Nous avons principalement utilisé les noms des fonctions depuis le manuel des "helpers" d'eBPF.

- **args :**

Ces arguments contiennent un nom et un type.

Les fonctions helpers d'eBPF dans le noyau Linux ne peuvent pas avoir plus de 5 arguments. De plus, certaines fonctions peuvent être appelées avec un nombre d'arguments non fixe. Par exemple, pour la fonction `bpf_printk` nous avons décidé de compter le nombre d'apparitions du symbole de format (%) dans la chaîne de caractères du premier argument afin de déterminer le nombre d'arguments.

- **content :**

Il s'agit du corps de la fonction, où l'implémentation réelle de la fonction est définie. Ce champ sera utilisé pour générer le corps des fonctions définies dans le code C. Pour notre projet, l'objectif étant uniquement de tester la sûreté d'un code eBPF avec MOPSA, il n'est pas nécessaire de reproduire fidèlement ces fonctions en C. Il suffit simplement de respecter les types de

retours de celles-ci. C'est pourquoi nous avons pris la décision de retourner des valeurs aléatoires via des fonctions MOPSA prédéfinies.

Un exemple de ce record est le suivant:

```
public static CFunction bpf_map_lookup_elem() {
    return new CFunction(CType.Void().ptr(),
        "bpf_map_lookup_elem",
        new CArg[]{
            new CArg("map", CType.BpfMap().ptr()),
            new CArg("key", CType.Void().ptr()),
        },
        "return _mopsa_rand_void_pointer();");
}
```

Il génère le code C suivant:

```
void* bpf_map_lookup_elem(struct bpf_map* map, void* key){
    return _mopsa_rand_void_pointer();
}
```

3.5. Problèmes rencontrés

3.5.1. Tableaux statiques

Un des sujets qui nous a posé des difficultés était les tableaux eBPF. Comme il n'y a pas d'instructions de déclaration d'une variable en eBPF nous ne savions pas comment déclarer une variable pour les tableaux avec le bon type et surtout, la bonne taille. Un autre point qui nous a perturbé était le fait que les offsets utilisés pour les tableaux étaient négatifs. De plus, dans un des exemples que nous avons choisi, la façon dont le code eBPF initialise les champs d'un tableau, était de les mettre à jour par blocs de 8 octets, alors que dans le code source, nous avons un tableau de caractères.

Voici un exemple du code généré pour initialiser un tableau :

```
L1: *((__u64*) (r10 + (-8))) = r1;
L2: *((__u64*) (r10 + (-16))) = r1;
L3: *((__u64*) (r10 + (-24))) = r1;
L4: *((__u64*) (r10 + (-32))) = r1;
...
```

Finalement, nous avons décidé d'utiliser un tableau de taille fixe : 512 caractères. Nous avons en effet pris en compte le fait que la taille de la pile de tous les

programmes eBPF est fixée à 512 octets. Nous avons également choisi de convertir ce tableau en pointeur de void. Nous n'y ajoutons pas toutes les variables de la pile de notre programme, mais seulement le retour des instructions qui utilisent le registre `r10`.

Pour les offsets négatifs, nous avons trouvé qu'il était commun pour les assembleurs d'utiliser des offsets à partir du haut de la pile, qui est à l'offset 0. Pour effectuer cette simulation, nous avons fait le choix de paramétrer la variable `r10`, qui sert de pointeur vers la pile, afin qu'elle pointe vers la fin de celle-ci plutôt que le début. Nous avons aussi compris que les mises à jour par blocs de 8 octets n'étaient pas la conséquence de la taille des données stockées dans le tableau, mais un raccourci permettant de diminuer le nombre d'instructions dans le code.

3.5.2. Maps eBPF

Un "map" eBPF est une structure de données partagée entre le code eBPF dans le noyau et le code de l'espace utilisateur. Ces maps peuvent être utilisées pour stocker des données, telles que des statistiques sur les appels système, qui peuvent être consultées et modifiées à la fois par le noyau et par l'espace utilisateur.

Les maps eBPF sont une partie intégrante des programmes eBPF car elles permettent une communication bidirectionnelle entre l'espace utilisateur et l'espace noyau. Cela permet une grande flexibilité dans ce qui est possible de faire avec eBPF. Par exemple, nous pourrions utiliser une map eBPF pour collecter des statistiques sur les appels système et ensuite consulter ces statistiques depuis l'espace utilisateur.

Il existe plusieurs types de maps eBPF, cela comprend des tables de hachage, des tableaux, des files d'attente et des piles. Chaque type de map a ses propres caractéristiques et utilisations préférées. Par exemple, une table de hachage peut être utilisée pour stocker les statistiques des appels système par PID, tandis qu'une file d'attente peut être utilisée pour communiquer des événements de l'espace noyau à l'espace utilisateur.

Les maps peuvent être décrites dans la section ".maps" du fichier ELF. Certaines maps viennent avec des valeurs d'initialisation stockées dans le fichier ELF (comme les string), d'autres ont leurs éléments initialisés à 0.

Mais ces maps ne sont utilisées qu'à partir de certaines fonctions helpers du noyau. Donc pour simuler le fonctionnement d'une map, nous avons créé ces fonctions dans notre bibliothèque. Ces fonctions ne sont pas de vrais équivalents des

fonctions dans le noyau, et elles retournent des valeurs aléatoires à l'aide des fonctions prédéfinies de MOPSA. En effet, retourner une valeur aléatoire permet de gérer le cas des maps dynamiques où le contenu est modifié pendant l'exécution du programme, par le programme eBPF lui-même, par le reste du noyau ou par un client en userland qui a également accès à cette map.

Au pire, dans le cas d'une map constante dont le contenu est connu et défini dans le fichier eBPF, la méthode proposée se traduit juste par une perte de précision, mais maintient la sûreté de l'analyse (pas d'erreur oubliée), mais cela suffit pour pouvoir tester ces fonctions avec MOPSA.

3.6. Préparation des exemples

Pour les exemples nous utilisons les programmes tests qui sont fournis par la bibliothèque de libbpf-bootstrap et aussi les programmes fournis dans le repository ebpf-samples. Tous les exemples que nous utilisons sont stockés dans le dossier `examples` de notre projet. Chaque exemple regroupe plusieurs fichiers, en plus du code source de ce dernier, qui sont listés ci-après :

- **elf:**

C'est le fichier elf du programme source. C'est essentiellement le fichier .o qui est généré lors de la compilation d'un programme eBPF.

- **sectionName:**

Il s'agit du nom de la section où le code eBPF de notre programme eBPF est situé.

- **strings:**

Il s'agit des chaînes de caractères de notre programme source. Ce fichier est généré à partir du script `extract\_string.sh` et est au format binaire. Au moment de la lecture de ce fichier, nous remplaçons le caractère `null` par le caractère `"\0"`. Ensuite dans le programme généré, toutes les chaînes de caractères du programme sont stockées dans une même variable, et séparées par le caractère `"\0"`. Les différentes chaînes sont accessibles en partant d'un offset différent partant du début du même tableau regroupant les chaînes.

Les trois fichiers ci-dessus sont les fichiers sources que notre traducteur prend en entrée.

En sortie, notre programme crée les 4 fichiers suivants:

- **"lib.c" et "lib.h":**

Ces fichiers contiennent la bibliothèque des fonctions que nous avons implémentées.

- **"prog.c":**

Ce fichier contient le code C compilé depuis le code du programme source.

- **"prog.h":**

Ce fichier contient les variables globales de notre programme si nécessaire. Il est aussi inclus dans "lib.c" au cas où les fonctions "helpers" en ont besoin aussi.

3.7. Exemple

Une manière de créer des programmes eBPF est de compiler un code C avec un compilateur C vers eBPF (d'autres langages que le C sont possibles, mais ce dernier reste le plus efficace). Toutefois, l'objectif est d'analyser le code eBPF après compilation, et pas le code source C original. Il existe par ailleurs d'autres méthodes permettant d'obtenir un code eBPF qu'à partir d'un code source C, pour lesquelles notre méthode reste applicable.

Il est important de faire la distinction entre le code source C, et le code C généré : le premier permet uniquement de générer des cas de tests eBPF qui nous seront utiles pour les tests de l'outil que nous développons, tandis que le second, est le code produit par notre convertisseur. Le but n'est pas de générer un code identique au code source, mais d'obtenir un code qui puisse être analysé par MOPSA.

Voici un exemple du code traduit par notre traducteur :

- **Code source en C:**

```
int func(void* ctx) {  
    char array[256] = "";  
    uint32_t rand32 = bpf_get_prandom_u32();  
    uint32_t index = *(unsigned char*)&rand32;  
    return array[index];  
}
```

- Code **eBPF** (représentation textuelle):

```

0:      r1 = 0
1:      *(u64 *) (r10 - 8) = r1
2:      *(u64 *) (r10 - 16) = r1
...
32:     *(u64 *) (r10 - 256) = r1
33:     r0 = get_prandom_u32:7()
34:     r0 &= 255
35:     r1 = r10
36:     r1 += -256
37:     r1 += r0
38:     r0 = *(u8 *) (r1 + 0)
39:     r0 <<= 56
40:     r0 >>= 56
41:     exit

```

- Code **C** généré:

```

int main() {
    __u64 r0;
    __u64 r1;

    char stack[512] = {0};
    r10 = (__u64) (stack + 511);
    L0: r1 = 0;
    L1: *((__u64*) (r10 + (-8))) = r1;
    L2: *((__u64*) (r10 + (-16))) = r1;
    ...
    L32: *((__u64*) (r10 + (-256))) = r1;
    L33: r0 = bpf_get_prandom_u32();
    L34: r0 &= -1;
    L35: r1 = r10;
    L36: r1 += -256;
    L37: r1 += r0;
    L38: r0 = *((__u64*) (r10 + (0)));
    L39: r0 <<= 56;
    L40: r0 >>= 56;
    L41: return r0;
}

```

On peut remarquer que nous avons beaucoup d'instructions de type :

```
*((__u64*) (r10 + (-8))) = r1
```

Malgré cela, on note qu'il n'y a pas de perte d'efficacité dans le traducteur, puisque ces instructions correspondent à des instructions originales du code eBPF (tel que généré par le compilateur). Le code C est en effet proportionnel au code eBPF source.

3.8. Analyse avec MOPSA

Une fois notre traducteur fonctionnel, l'étape suivante de notre projet fut d'analyser le code C produit par celui-ci à l'aide de MOPSA. Il nous fallait donc trouver de bons exemples de code à traduire et analyser.

3.8.1. Banc de tests

Nous avons inclus à notre projet un certain nombre de codes eBPF source à convertir et analyser à l'aide de MOPSA. Bien que la plupart aient été récupérés du projet Prevail mentionné plus haut, d'autres ont par exemple été produits par nous-même. Voici un tableau donnant des détails sur ceux-ci :

Nom	Taille (nombre instructions)	Origine
byteswap	34	Projet Prevail
factoriel	15	Original
map_in_map	19	Projet Prevail
twomaps	15	Projet Prevail
stackok	42	Projet Prevail
twostackvars	57	Projet Prevail
twotypes	23	Projet Prevail

Les fichiers byteswap et factoriel effectuent des opérations simples sur des entiers. map_in_map et twomaps opèrent sur des maps, [stackok](#) et [twostackvars](#) sur des tableaux, et [twotypes](#) regroupe les deux structures de données précédentes dans un seul et même exemple.

3.8.2. Analyse

Après avoir été traduits par notre convertisseur en C, ces tests ont été fournis à l'outil MOPSA afin d'être analysés.

Nom	Warnings	Type de warning	Erreurs	Commentaires
byteswap	4	Integer overflow	0	
factoriel	4	Stub condition, Integer overflow	0	
map_in_map	6	Invalid memory access, Integer overflow	0	2 suppositions
twomaps	4	Invalid memory access, Integer overflow	0	1 suppositions
stackok	35	Invalid memory access, Integer overflow	0	32 suppositions
twostackvars	21	Invalid memory access, Integer overflow	0	17 suppositions
twotypes	6	Invalid memory access, Integer overflow	0	2 suppositions

Pour les pointeurs de la pile, MOPSA ignore les modifications faites par les instructions qui utilisent cette dernière. Elle ignore donc les modifications faites par ces instructions et ne les vérifie pas. Nous pensons que cela est dû au fait que nous faisons de l'arithmétique sur les pointeurs pour ces instructions. Les warnings de type "Invalid memory access" et les suppositions faites par MOPSA sont dûes à cela.

Les warnings de type Integer overflow sont, quant à eux, dûs aux incompatibilités d'entiers. Nous avons pensé à convertir tous les entiers vers des entiers non signés 64 bits (`__u64`) mais cela n'était pas une bonne solution, car les instructions de chargement de mémoire en eBPF sont de différentes tailles. Par exemple, la valeur binaire de l'entier -1 n'est pas la même lors d'un chargement de 8 bits et 64 bits. Il était donc nécessaire de faire cette conversion au moment du chargement de la mémoire. Il pourrait être intéressant de prendre en charge cette différence dans une future version du projet.

4. Réalisations additionnelles

4.1. Prise en charge des structures courantes

Une première idée d'ajout à notre projet, qui pourrait nous permettre d'analyser des programmes eBPF plus variés serait de prendre en charge des structures de données plus variées, et de façon plus approfondie. En effet, on peut remarquer, en consultant le manuel eBPF, et différents exemples de codes dans d'autres projets, qu'il est assez courant de se servir de certaines structures de données prédéfinies et même que certaines d'entre elles sont redondantes.

Les maps, sont un bon exemple de structures utilisées assez souvent. En effet, notre projet prend bel et bien en charge celles-ci, mais de façon assez superficielle. Comme expliqué dans une partie précédente, les maps de notre projet correspondent à un simple type, dont les valeurs sont simulées comme étant des pointeurs de void aléatoires. Cependant, il serait intéressant d'élargir cette définition, afin de permettre aux maps d'avoir des valeurs possédant le bon type de pointeur.

On peut également donner l'exemple d'une structure nommée `sk_buff`, qui correspond à des paquets réseaux du noyau Linux. On remarque, notamment dans le manuel bpf, que cette structure est très souvent utilisée dans les différentes fonctions prédéfinies d'eBPF. Ainsi, la prise en charge de structures courantes comme celle-ci, pourrait permettre d'étendre la bibliothèque en y définissant plus de fonctions, et d'analyser des codes eBPF plus variés à l'aide de MOPSA.

4.2. Amélioration des tableaux eBPF

Les tableaux, ou array, sont une structure de données intéressante, permettant de stocker des valeurs dans des espaces mémoires consécutifs prédéfinis lors de leur déclaration.

Comme nous l'avons déjà mentionné, ces structures sont prises en charge dans notre projet même si leur utilisation dans le code eBPF source reste assez limitée. En effet, le type d'un tableau, et sa taille n'étant pas explicités en eBPF, il n'est pas possible de déterminer à l'avance ces propriétés. C'est pourquoi nous avons fait le choix de définir une taille constante pour nos array, et d'utiliser un type générique.

Ces différents points pourraient être améliorés. Cependant, il est à remarquer que

ceci implique qu'il faille examiner le code se situant après la définition d'un tableau notamment pour le type. En effet, il pourrait être suffisant d'examiner l'affectation d'une case du tableau afin d'en déterminer le type, ou, à défaut, de lui donner un type générique. De même concernant la taille, il faudrait examiner les indices du tableau utilisés et choisir une taille en fonction du plus grand indice utilisé, ou une taille par défaut sinon (aussi petite soit-elle).

5. Perspectives d'améliorations pour le futur

Ce projet étant ambitieux, il nous est impossible de le traiter en profondeur durant le peu de temps dont nous disposons. Cependant, avec ce que nous avons déjà produit, nous pouvons donner quelques idées de points à aborder afin de l'enrichir.

5.1. Pointeurs

Les pointeurs constituent un point de notre projet sur lequel nous ne nous sommes pas attardés. En effet, la prise en charge que nous proposons reste assez limitée et est sujet à amélioration. En effet, dans notre bibliothèque, une unique fonction retournant un pointeur est prise en charge. Il s'agit de la fonction `lookup` dans un `map`, qui pour une clé et une `map` données, retourne la valeur correspondante. Cette fonction n'est pas restrictive, puisqu'elle retourne un pointeur du type générique (`void`), ce qui nous permet de simuler celle-ci sans prendre de risques.

Une solution que nous pourrions proposer serait de créer au préalable des variables de pointeurs pour chaque type, afin d'avoir des pointeurs prédéfinis à utiliser et retourner, ce qui permettrait de simuler plus rigoureusement le code eBPF à traduire.

5.2. Gestion de la taille des entiers

Dans le cadre de ce projet, nous avons convenu que les deux familles d'instructions ALU 32 et ALU 64 (Arithmetic Logic Unit ou Unité de Logique Arithmétique) seraient traitées en générant un code identique. Ce choix stratégique découle de notre décision initiale de représenter les registres en tant que variables de type `uint64_t`.

Étant donné que ce type de données est suffisamment grand pour englober toutes les opérations ALU 64 et ALU 32, nous avons jugé inutile de convertir ces opérations vers des types plus petits ou de limiter les bornes des variables pour les opérations

32 bits de la famille ALU 32. Ainsi, nous avons opté pour une approche simplifiée et plus efficace pour gérer ces instructions.

Bien que notre équipe de projet ait choisi de traiter de manière uniforme les instructions des familles ALU 32 et ALU 64 par l'utilisation de variables de type `uint64_t`, il est important de noter que cette décision n'est pas irréversible. Les équipes futures travaillant sur ce projet peuvent choisir de remodeler cette approche si nécessaire.

Si les exigences ou les objectifs du projet changent et qu'un traitement distinct de ces deux types d'opérations devient avantageux ou nécessaire, cette stratégie peut être adaptée. Le code peut être révisé pour permettre un traitement spécifique à chaque famille d'instructions, introduisant ainsi des optimisations ou des améliorations diverses.

5.3. Étendre la bibliothèque

Enfin, la dernière idée que nous allons aborder ici concerne la bibliothèque. En effet, nous avons donné différents points d'améliorations ajoutant la prise en charge de nouvelles fonctionnalités pour ce projet. Bien sûr, et nous l'avons également mentionné si nécessaire, ces changements impliquent, pour la plupart, une prise en charge plus large des fonctions natives disponibles dans le manuel d'eBPF. Ainsi, de nombreuses fonctions, jusqu'ici incompatibles avec notre traducteur, le deviennent, et cela permet ainsi de l'enrichir toujours plus, petit à petit, jusqu'à prendre en charge la quasi-totalité de celui-ci.

Un autre point intéressant concernant la bibliothèque serait d'y ajouter le corps de fonctions, et non seulement s'assurer de leur type de retour. On pourrait également, de façon alternative, établir un ensemble d'assertions vérifiant que la fonction appelée a bien remplie sa tâche. Cette idée contribuerait à rendre la bibliothèque plus sûre, et ajouterait une couche supplémentaire de sûreté à ce programme, en amont de l'analyse statique effectuée par MOPSA.

Ces différentes idées permettraient donc à cette bibliothèque d'être plus complète et rigoureuse, assurant ainsi la prise en charge de la totalité des fonctionnalités d'eBPF, et permettant d'atteindre l'apogée de ce projet.

6. Conclusion

Le projet de traduction et de vérification des programmes eBPF avec MOPSA a atteint plusieurs jalons importants. Nous avons réussi à développer un traducteur fonctionnel qui convertit le code-octet eBPF en C, ainsi qu'une bibliothèque C qui simule l'environnement d'exécution des programmes eBPF. Ces outils ont permis de réaliser une analyse statique de plusieurs programmes eBPF, identifiant des erreurs potentielles et augmentant ainsi la sûreté de leur exécution dans le noyau Linux.

Bien que des progrès significatifs aient été réalisés, le projet reste ouvert à des améliorations futures. Les futurs travaux pourraient inclure l'extension de la bibliothèque pour couvrir plus de fonctionnalités eBPF, l'amélioration de la précision du traducteur pour réduire les fausses alarmes générées par MOPSA, et l'adaptation de l'outil pour supporter d'autres langages de programmation utilisés dans les environnements de noyau.

En conclusion, ce projet a non seulement renforcé notre compréhension des défis liés à la programmation au niveau du noyau mais a également posé les bases pour des recherches futures qui pourraient mener à des avancées significatives dans le domaine de la sécurité des systèmes d'exploitation.

7. Bibliographie

“libbpf: Automated upstream mirror for libbpf stand-alone build.” *GitHub*,
<https://github.com/libbpf/libbpf>

“libbpf-bootstrap: Scaffolding for BPF application development with libbpf and BPF CO-RE.”
GitHub, <https://github.com/libbpf/libbpf-bootstrap>

“bpftool: Automated upstream mirror for bpftool stand-alone build.” *GitHub*,
<https://github.com/libbpf/bpftool>

“bpf.h du noyau Linux.” *GitHub*,
<https://github.com/torvalds/linux/blob/master/include/uapi/linux/bpf.h>

“ebpf-disasm : Simple ebpf disassembler in Rust.” *GitHub*,
<https://github.com/badboy/ebpf-disasm>

“ebpf-samples: Sample ebpf programs to analyze.” *GitHub*,
<https://github.com/vbpf/ebpf-samples>

PREVAIL: a new eBPF verifier | PREVAIL is a new verifier for eBPF programs.
<https://vbpf.github.io>